

Engineering Statistics

Yili Hong

Chapter 13: Machine Learning and Prediction Methods

Outline

Regression Trees

Classification Trees

Random Forests

Boosting, Gradient Boosting, and XGBoost

Introduction to Regression Trees

- ▶ Regression trees divide the predictor space into a number of simple regions.
- ▶ The goal is to predict a continuous response variable Y .
- ▶ The model takes the form:

$$y_i = \sum_{j=1}^m \beta_j \mathbf{1}(x \in R_j)$$

where $R_1, \dots, R_m$ are regions of the predictor space.

- ▶ We can easily get $\hat{\beta}_j = \bar{y}_j$. Within each region, the prediction is a constant equal to the average of the training responses in that region.

Example of Partitions

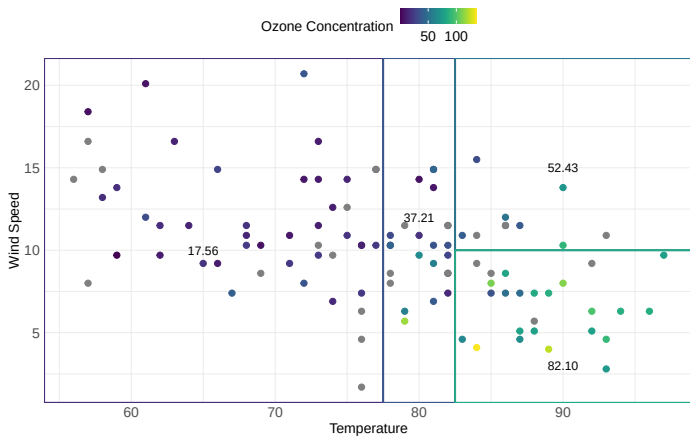


Figure: The four-region partition of the Air Quality data from the regression tree.

Bias-Variance Trade off

- ▶ For a new observation y in R_j , the mean square error for prediction is:

$$\text{MSPE} = \text{Var}(y) + \text{Var}(\hat{\beta}_j) + \text{bias}^2 \quad (1)$$

$$= \sigma_j^2 + \frac{\sigma_j^2}{n_j} + \left(E(y) - \frac{\sum_{i:x \in R_j} E(y_i)}{n_j} \right)^2 \quad (2)$$

- ▶ The bias will be small if $E(y)$ changes little across R_j , indicating more regions leading to smaller bias.
- ▶ However, the variance increases as the increase of the number of regions.

Recursive Binary Splitting

- ▶ For each predictor X_j and possible cutpoint s , define regions:

$$R_l(t, s) = \{x : x_t < s\}, \quad R_u(t, s) = \{x : x_t \geq s\}$$

- ▶ The best split minimizes SSE:

$$SSE(t, s) = \sum_{i: x_i \in R_l(t, s)} (y_i - \bar{y}_{R_l})^2 + \sum_{i: x_i \in R_u(t, s)} (y_i - \bar{y}_{R_u})^2$$

- ▶ Because exhaustive search is computationally infeasible, a greedy algorithm called recursive binary splitting is used.
- ▶ At each step, the predictor and split point among all regions that achieve the smallest SSE are chosen.

Tree Growth and Stopping

- ▶ The process continues until a stopping criterion is met, such as:
 - ▶ A minimum number of observations per node.
 - ▶ A minimum reduction of variance.
- ▶ Fully grown trees can overfit and lead to high variance.
- ▶ To address this, trees are often pruned using a cost-complexity measure.

Cost-Complexity Pruning

- ▶ For a subtree $T \subset T_0$ (where T_0 is the full tree), define:

$$R_\alpha(T) = \sum_{j=1}^{|T|} \sum_{i: x_i \in R_j} (y_i - \bar{y}_j)^2 + \alpha |T|$$

- ▶ $|T|$ is the number of terminal nodes, and α is a tuning parameter controlling model complexity.
- ▶ For each α , there is a corresponding subtree T_α that minimizes $R_\alpha(T)$. But due to computation cost, similar greedy method is used to find the pruning tree.
- ▶ Larger α means more pruning and $\alpha = 0$ means no pruning.

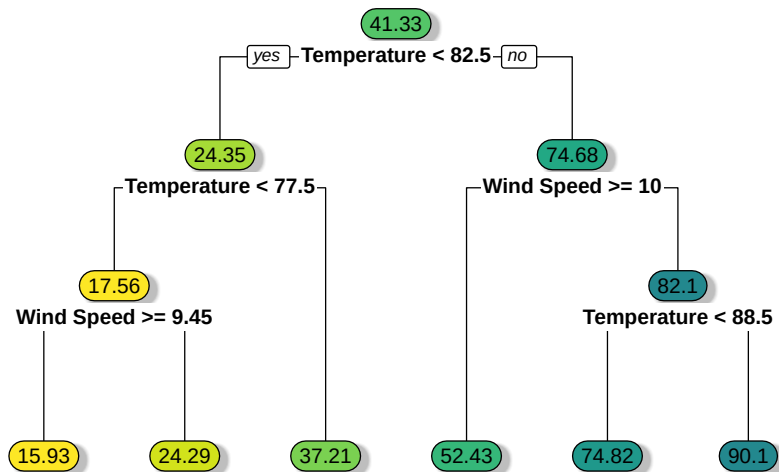
Cost-Complexity Pruning

- ▶ Equivalent practical parameter: complexity parameter CP (often used in software) is defined as: $\frac{\alpha}{R_0}$. Stop pruning when

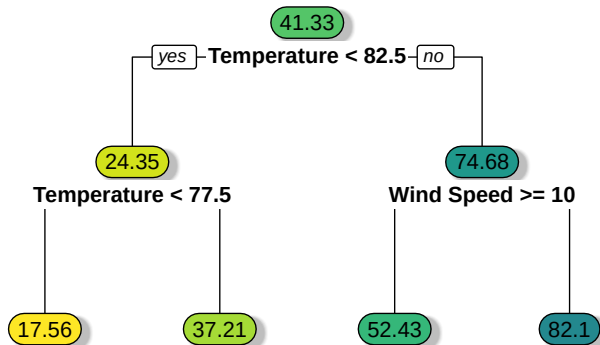
$$\frac{R(T_1) - R(T)}{R_0} > \frac{\alpha}{R_0} = \text{CP}$$

- ▶ Cross-validation is used to select the optimal α (CP).

Example: Ozone Data, $CP = 0.001$



Example: Ozone Data, $CP = 0.05$



Example: Ozone Data

- ▶ A regression tree can be used to predict ozone concentration using predictors such as temperature and wind.
- ▶ As α increases, the tree becomes smaller.
- ▶ Larger trees (small α) have lower bias but higher variance.
- ▶ Smaller trees (large α) have higher bias but lower variance.

Tree Pruning with 10-fold Cross Validation

- ▶ We select the **simplest pruned tree** whose CV error is close to the minimum.
- ▶ This corresponds to the tree with **3 splits (4 leaves)** shown in the figure above.
- ▶ On the test data: the large tree (b) with $CP = 0.001$ has $MSPE = 27.763$, while the pruned tree (a) has $MSPE = 27.375$.
- ▶ The simpler pruned tree performs slightly better, likely because the larger tree suffers from **overfitting**.

Feature Space Partitions from the Pruned Tree

The pruned tree divides the Air Quality feature space into **four regions**:

$$\mathcal{R}_1 = \{\mathbf{x} : \text{Temperature} < 77.5\},$$

$$\mathcal{R}_2 = \{\mathbf{x} : 77.5 \leq \text{Temperature} < 82.5\},$$

$$\mathcal{R}_3 = \{\mathbf{x} : \text{Temperature} \geq 82.5, \text{Wind Speed} \geq 10\},$$

$$\mathcal{R}_4 = \{\mathbf{x} : \text{Temperature} \geq 82.5, \text{Wind Speed} < 10\}.$$

- ▶ Each region corresponds to one **leaf node** of the pruned tree.
- ▶ The splits are determined by thresholds on **Temperature** and **Wind Speed**.

Interpretation of Regression Trees

- ▶ Each split represents a decision rule on a predictor variable.
- ▶ The final model is interpretable as a set of if-then rules.
- ▶ Regression trees capture interactions automatically.
- ▶ However, small changes in data can lead to different splits, showing instability.

Introduction to Classification Trees

- ▶ Classification trees are used when the response variable Y is categorical. The k categories are labeled $1, 2, \dots, k$.
- ▶ The feature space is partitioned into regions $R_1, \dots, R_m$.
- ▶ The model takes the form:

$$\Pr(y = l|x) = \sum_{j=1}^m p_{jl} \mathbf{1}(x \in R_j)$$

- ▶ p_{jl} is the probability that a response with predictors in R_j falls in category l . Also, $0 \leq p_{jl} \leq 1$ and $\sum_{l=1}^k p_{jl} = 1$

Introduction to Classification Trees

- ▶ A straightforward estimate of p_{jl} is:

$$\hat{p}_{jl} = \frac{1}{n_j} \sum_{x_i \in R_j} \mathbf{1}(y_i = l)$$

- ▶ For a new observation, the predicted class is the majority class in the corresponding region.

Splitting Criteria and Construction of a Classification Tree

- ▶ Instead of minimizing SSE, we minimize measures of node impurity.
- ▶ Common impurity measures:

$$\text{Gini Index: } \sum_l \hat{p}_{jl}(1 - \hat{p}_{jl})$$

$$\text{Entropy: } - \sum_l \hat{p}_{jl} \log(\hat{p}_{jl})$$

- ▶ Tree construction is the same as in regression tree by replacing SSE with Gini index or Entropy
- ▶ Splits chosen to reduce node impurity. Gini and entropy give similar behavior in practice.

Tree Pruning and Performance

- ▶ The cost-complexity pruning method also applies to classification trees.
- ▶ Cross-validation is used to choose α that yields the best classification accuracy.
- ▶ Classification trees are interpretable and easy to visualize.

Example: Classification Tree

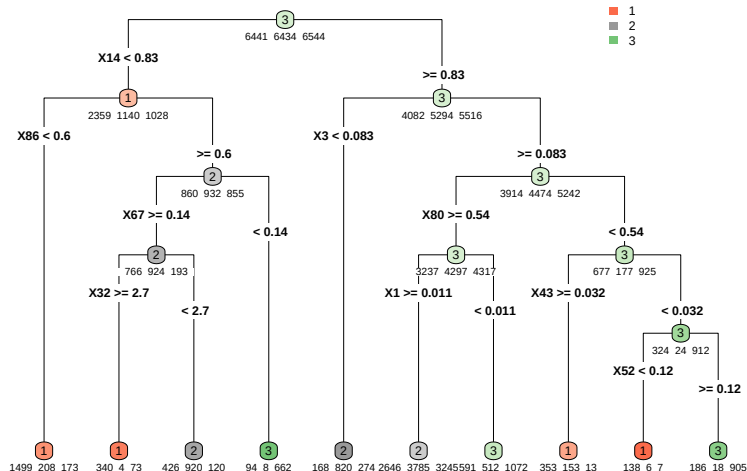


Figure: Classification tree for KEGG NEPDF data

Example: Classification Tree

- ▶ Classification trees can be used for categorical responses such as “Low,” “Medium,” and “High.”
- ▶ Each terminal node provides a predicted class and class probabilities.
- ▶ Visualization helps to interpret relationships among predictors and classes.

Advantages and Limitations

- ▶ Advantages:
 - ▶ Simple and interpretable.
 - ▶ Can handle qualitative predictors without preprocessing.
- ▶ Limitations:
 - ▶ High variance.
 - ▶ Can be unstable with small data changes.

Motivation for Random Forests

- ▶ Single trees are easy to interpret but have high variance.
- ▶ Bootstrap repeatedly samples from the origin dataset with replacement. It allows for statistical inference without making strong assumption about the data's distribution
- ▶ Bagging reduces variance by averaging multiple trees trained on bootstrap samples.
- ▶ Random forests further decorrelate trees by using random subsets of predictors at each split.

Algorithm Overview

- ▶ Assume we have n observations and we want to construct m trees in total. The total number of predictors is p .
- ▶ For $b = 1, \dots, f$:
 1. Draw a bootstrap sample of the training data.
 2. Grow a tree without pruning.
 3. At each split, select a random subset of predictors to consider.
- ▶ The size of random subset of predictors p_0 is commonly chosen as $\sqrt{p}$. When p is small, we chose $p_0 = p$.

Bootstrap aggregation(bagging)

- ▶ If we want to estimate y given x , we do in the following steps.
 1. Form $\hat{y}_j = T_j(x)$ for $j = 1, \dots, f$.
 2. If outcome is continuous, calculate $\hat{y} = \sum_{j=1}^f \frac{\hat{y}_j}{f}$. If outcome is categorical, get $\hat{y}$ by majority vote.
- ▶ Bagging doesn't produce partitions of the feature space.
- ▶ In the cost of interpretability, bagging effectively reduces variance.

Properties of Random Forests

- ▶ Effective in reducing variance while maintaining low bias.
- ▶ Built-in measure of prediction error via out-of-bag (OOB) samples.
- ▶ Provide variable importance measures based on prediction error or Gini decrease.

The Gini Index in Random Forests

- ▶ For classification trees, node impurity is measured by the Gini Index.
- ▶ We denote the proportion of class c in node d as $\hat{p}_{c,d}$,

$$Gini(d) = \sum_{c=1}^C \hat{p}_{c,d}(1 - \hat{p}_{c,d}) = 1 - \sum_{c=1}^C \hat{p}_{c,d}^2.$$

- ▶ The Gini Index is small when a node is “pure,” meaning most observations belong to one class.
- ▶ At each split, Random Forests choose the variable and threshold that yield the largest decrease in Gini impurity.

Decrease in Gini Impurity

- ▶ Suppose a node d is split into left ($d_l(j)$) and right ($d_r(u)$) child nodes.
- ▶ The decrease in Gini impurity is:

$$\Delta G(x_j|d) = Gini(d) - [p_l^{(j)} Gini(d_l^{(j)}) + p_u^{(j)} Gini(d_r^{(j)})]$$

- ▶ $p_l^{(j)}$ and $p_u^{(j)}$ are the proportions of observations from node d that fall in $d_l^{(j)}$ and $d_r^{(j)}$ respectively.
- ▶ The split maximizing $\Delta Gini$ is selected.
- ▶ This criterion is used for every split in every tree.

Total Gini Reduction for Variable x_j

For a given tree T , let $\mathcal{D}_T$ be the set of internal (non-leaf) nodes. If n_d is the number of training samples in node d , and N the total number of samples, then the total Gini reduction from variable x_j is:

$$\Delta G_T(x_j) = \sum_{d \in \mathcal{D}_T, v(d)=x_j} \frac{n_d}{N} \Delta G(x_j \mid d) \quad (3)$$

- ▶ $v(d)$: variable used to split node d .
- ▶ $\frac{n_d}{N}$: weight reflecting node size.
- ▶ $\Delta G(x_j \mid d)$: Gini decrease at node d .

Mean Decrease in Gini (MDG) in Random Forests

For a random forest with f trees $T_1, \dots, T_f$, the overall mean decrease in Gini (MDG) for variable x_j is the average Gini reduction contributed by x_j across all trees:

$$\text{MDG}(x_j) = \frac{1}{f} \sum_{k=1}^f \Delta G_{T_k}(x_j) \quad (4)$$

- ▶ MDG measures each variable's average contribution to reducing overall prediction impurity.
- ▶ A larger $\text{MDG}(x_j)$ means x_j often produces splits that strongly reduce the Gini index.

Summary of Random Forests and Gini

- ▶ Random Forests aggregate many trees built on random subsets of data and features.
- ▶ The Gini Index quantifies impurity and guides each split decision.
- ▶ Total Gini reduction provides a natural measure of variable importance.
- ▶ Random forests achieve high predictive accuracy and can efficiently handle large, high-dimensional datasets.
- ▶ However, they are less interpretable than single trees and require greater computational resources.

From Random Forests to Boosting

- ▶ Random forests combine multiple independent trees to reduce variance by averaging.
- ▶ In contrast, boosting builds trees sequentially, each new tree correcting the errors of the previous ones.
- ▶ The goal is to reduce prediction bias by focusing on hard-to-predict observations.

Concept of Boosting

- ▶ Boosting constructs a strong model by combining multiple weak learners.
- ▶ Each new tree focuses on the residuals or misclassified points from the prior model.
- ▶ The final prediction is a weighted sum of all weak learners:

$$\hat{f}(x) = \sum_{m=1}^M \alpha_m h_m(x)$$

- ▶ Common base learners: shallow decision trees.

AdaBoost: Early Form of Boosting

- ▶ Proposed by Freund and Schapire (1997) for classification.
- ▶ Reweights training samples: misclassified observations receive higher weights.
- ▶ Sequentially trains weak classifiers to minimize classification errors.
- ▶ Produces a strong classifier that combines many simple learners.

$$\hat{f}(x) = \sum_{m=1}^M \alpha_m h_m(x),$$

Gradient Boosting: General Framework

- ▶ Generalizes boosting to any differentiable loss function.
- ▶ Each iteration fits a new learner to the negative gradient of the loss (the pseudo-residuals).
- ▶ The model is updated additively:

$$f_m(x) = f_{m-1}(x) + \alpha_m h_m(x)$$

- ▶ Gradient boosting provides flexibility and strong predictive accuracy.

XGBoost: eXtreme Gradient Boosting

- ▶ A refined implementation of gradient boosting that adds:
 - ▶ **Regularization** to penalize complex trees.
 - ▶ **Shrinkage (learning rate)** to control update size.
 - ▶ **System-level optimizations** for speed and scalability.
- ▶ Efficient for large-scale and high-dimensional datasets.
- ▶ Widely used in practice for both regression and classification tasks.